

Programming In-Storage Computing with Located, Stateful Dataflow

<https://arxiv.org/abs/2609.19206>

1. Problem

- 문제: In-Storage Computing(ISC)은 데이터를 스토리지에서 처리해 데이터 이동을 줄이지만, 멀티스테이지 워크로드의 데이터 위치·상태·실행 흐름을 통합적으로 표현하고 최적화하기 어렵다.
- 왜 중요한가: ISC의 성능은 단순 연산 offloading보다 데이터 이동과 상태 관리를 얼마나 효율적으로 줄이느냐에 크게 좌우된다.

2. Method

- 핵심 아이디어: Epic — 데이터의 위치(location)와 상태/lifetime을 dataflow에 직접 표현하는 ISC 프로그래밍 시스템.
- 핵심 구성:
 - Location type: 데이터가 어디에 존재하는지 표현
 - Dataflow: 중간 결과와 데이터 lifetime을 표현
 - **keep**: invocation 간 필요한 state를 유지
- 기존 방법과 차이: 개별 연산을 스토리지로 offload하는 데 그치지 않고, 데이터 이동 + 연산 + 상태 + I/O-compute overlap을 workflow 전체에서 최적화한다.
- Compiler + Runtime: compiler가 data placement/fusion 등을 최적화하고, runtime이 비동기 실행과 device-resident state를 관리한다.

3. Experiments & Results

- 12개 workload 평가
 - File scanning
 - Database

- Machine learning
- 기존 5개 ISC 시스템 중 가장 강한 baseline 대비 평균 1.6× 성능 향상
- Host 기반 실행 대비:
 - 평균 4.2× speedup
 - 최대 16.1× speedup
- Application-side code는 최대 14× 감소
- 데이터 이동과 state management까지 함께 최적화하면서 기존 ISC보다 큰 성능 이득을 얻음.

4. Insight

- 핵심 기여: ISC를 단순한 “연산 offloading”이 아니라, 데이터의 위치·수명·상태를 포함한 전체 dataflow를 프로그래밍하고 최적화하는 문제로 확장했다.
- 한계점:
 - 다양한 실제 CSD/NVMe 환경에서의 일반화 가능성은 추가 검증이 필요하다.
 - 새로운 programming abstraction과 compiler/runtime을 도입해야 한다.
- 내가 가져갈 포인트:
 - 핵심 질문을 “연산을 어디서 실행할 것인가?” → “데이터가 어디에 있고, 언제 이동하며, 얼마나 오래 유지되어야 하는가?”로 바꾼 것이 중요하다.
 - Location + Lifetime + State를 프로그래밍 모델에 노출하고 compiler/runtime이 최적화한다는 구조가 핵심이다.